

Dataset loading and exploration: Python code

Exact source from the executed main.py; left-hand numbers identify original source lines.

```
74 def task_one_load_and_explore(path):
75     print("\nTASK 1: Dataset loading and initial exploration")
76     if not path.is_file():
77         raise FileNotFoundError(f"Dataset not found: {path}. Supply the CSV using -
-data.")
78     frame = pd.read_csv(path)
79     if frame.empty:
80         raise ValueError("The dataset contains no records.")
81     print("\nFirst 10 records:\n", frame.head(10).to_string(index=False))
82     print("\nLast 8 records:\n", frame.tail(8).to_string(index=False))
83     print("\nDataset shape:", frame.shape)
84     print("\nColumn names:", frame.columns.tolist())
85     print("\nData types:\n", frame.dtypes.to_string())
86     numeric = frame.select_dtypes(include="number")
87     print("\nNumerical summaries:\n", numeric.describe().to_string() if not numeri
c.empty else "No numerical columns.")
88     return frame
```

Exported execution evidence · main.py

Screenshot 1 of 27 · Generated from the executed source and saved run artifacts

First and last records: column band 1/3

The same required records are shown in column bands so every original column remains readable.

First 10 records

Comment_ID	Comment_Text	Toxicity_Label
COM000001	The message targets a group unfairly and violates community rules.	Identity_Attack
COM000002	Thank you for sharing this perspective with the community.	Clean
COM000003	That response is aggressive and not helpful.	Toxic
COM000004	I appreciate your explanation and the discussion was helpful.	Clean
COM000005	I respectfully disagree but I understand your point.	Clean
COM000006	That response is aggressive and not helpful.	Toxic
COM000007	I respectfully disagree but I understand your point.	Clean
COM000008	I appreciate your explanation and the discussion was helpful.	Clean
COM000009	This article provides useful information and clear examples.	Clean
COM000010	I appreciate your explanation and the discussion was helpful.	Clean

Last 8 records

Comment_ID	Comment_Text	Toxicity_Label
COM029993	Your message is unnecessarily hostile and offensive.	Toxic
COM029994	I appreciate your explanation and the discussion was helpful.	Clean
COM029995	Thank you for sharing this perspective with the community.	Clean
COM029996	This article provides useful information and clear examples.	Clean
COM029997	I respectfully disagree but I understand your point.	Clean
COM029998	I appreciate your explanation and the discussion was helpful.	Clean
COM029999	This article provides useful information and clear examples.	Clean
COM030000	I appreciate your explanation and the discussion was helpful.	Clean

First and last records: column band 2/3

The same required records are shown in column bands so every original column remains readable.

First 10 records

Comment_ID	Clean	Toxic	Severe_Toxic	Threat	Identity_Attack	Toxicity_Score	Comment_Length	Word_Count
COM000001	0	1	0	0	1	0.8180	66	10
COM000002	1	0	0	0	0	0.0690	58	9
COM000003	0	1	0	0	0	0.6220	44	7
COM000004	1	0	0	0	0	0.0100	61	9
COM000005	1	0	0	0	0	0.1400	52	8
COM000006	0	1	0	0	0	0.7290	44	7
COM000007	1	0	0	0	0	0.1930	52	8
COM000008	1	0	0	0	0	0.1830	61	9
COM000009	1	0	0	0	0	0.1420	60	8
COM000010	1	0	0	0	0	0.1530	61	9

Last 8 records

Comment_ID	Clean	Toxic	Severe_Toxic	Threat	Identity_Attack	Toxicity_Score	Comment_Length	Word_Count
COM029993	0	1	0	0	0	0.5200	52	7
COM029994	1	0	0	0	0	0.1950	61	9
COM029995	1	0	0	0	0	0.1740	58	9
COM029996	1	0	0	0	0	0.0130	60	8
COM029997	1	0	0	0	0	0.0500	52	8
COM029998	1	0	0	0	0	0.0780	61	9
COM029999	1	0	0	0	0	0.0030	60	8
COM030000	1	0	0	0	0	0.0820	61	9

First and last records: column band 3/3

The same required records are shown in column bands so every original column remains readable.

First 10 records

Comment_ID	Platform	Topic	Moderation_Priority	Language
COM000001	Community Chat	Sports	High	English
COM000002	News Comments	Entertainment	High	English
COM000003	Social Media	Politics	Low	English
COM000004	Social Media	Entertainment	Low	English
COM000005	Blog	Sports	Low	English
COM000006	Social Media	Technology	Low	English
COM000007	Blog	Entertainment	Low	English
COM000008	Forum	Gaming	Low	English
COM000009	Social Media	Education	Medium	English
COM000010	Forum	Entertainment	Low	English

Last 8 records

Comment_ID	Platform	Topic	Moderation_Priority	Language
COM029993	Forum	Gaming	Medium	English
COM029994	News Comments	Sports	Medium	English
COM029995	Community Chat	Gaming	Low	English
COM029996	Community Chat	Sports	Medium	English
COM029997	Blog	Politics	Medium	English
COM029998	Forum	Sports	Low	English
COM029999	Social Media	Sports	Medium	English
COM030000	Community Chat	Education	High	English

Dimensions, column names and data types

Dataset shape: 30,000 rows × 15 columns.

Column	Data type
Comment_ID	object
Comment_Text	object
Toxicity_Label	object
Clean	int64
Toxic	int64
Severe_Toxic	int64
Threat	int64
Identity_Attack	int64
Toxicity_Score	float64
Comment_Length	int64
Word_Count	int64
Platform	object
Topic	object
Moderation_Priority	object
Language	object

Numerical statistical summary (1)

Pandas describe() reports count, mean, standard deviation, minimum, quartiles and maximum.

	Clean	Toxic	Severe_Toxic	Threat	Identity_Attack
count	30000.0000	30000.0000	30000.0000	30000.0000	30000.0000
mean	0.6193	0.3807	0.0600	0.0486	0.0489
std	0.4856	0.4856	0.2376	0.2151	0.2157
min	0.0000	0.0000	0.0000	0.0000	0.0000
25%	0.0000	0.0000	0.0000	0.0000	0.0000
50%	1.0000	0.0000	0.0000	0.0000	0.0000
75%	1.0000	1.0000	0.0000	0.0000	0.0000
max	1.0000	1.0000	1.0000	1.0000	1.0000

Numerical statistical summary (2)

Pandas describe() reports count, mean, standard deviation, minimum, quartiles and maximum.

	Toxicity_Score	Comment_Length	Word_Count
count	30000.0000	30000.0000	30000.0000
mean	0.3245	56.3576	8.3406
std	0.3029	6.9626	0.9721
min	0.0000	39.0000	6.0000
25%	0.0790	52.0000	8.0000
50%	0.1610	58.0000	8.0000
75%	0.6240	61.0000	9.0000
max	0.9500	71.0000	11.0000

Cleaning and data quality: Python code (1/2)

Exact source from the executed main.py; left-hand numbers identify original source lines.

```

106 def clean_text(value):
107     text = unicodedata.normalize("NFKC", html.unescape(str(value))).lower()
108     text = re.sub(r"<[^>]*>", " ", text)
109     text = re.sub(r"https?:\/\/\S+|www\.\S+", " ", text)
110     text = "".join(character if character.isalpha() or character.isspace() else " "
    for character in text)
111     return " ".join(text.split())

114 def task_two_prepare(frame, arguments, output):
115     print("\nTASK 2: Cleaning, missing values, duplicates, and target analysis")
116     missing = frame.isna().sum().rename("missing_count")
117     print("\nMissing values per column:\n", missing.to_string())
118     missing.to_csv(output / "missing_values.csv")
119     duplicates = frame[frame.duplicated(keep=False)]
120     print(f"\nExact duplicate records beyond the first occurrence: {int(frame.duplicated().sum())}")
121     print(f"Rows participating in exact duplicates: {len(duplicates)}")
122     print(duplicates.to_string(index=False) if not duplicates.empty else "No exact duplicate records.")
123     duplicates.to_csv(output / "duplicate_records.csv", index=False)
124     text_column = select_column(frame, arguments.text_column, {"comment_text", "text", "comment", "comments", "content"}, "--text-column")
125     flag_names = {"toxic", "severe_toxic", "obscene", "threat", "insult", "identity_hate"}
126     flags = [column for column in frame.columns if normalized_name(column) in flag_names]
127     target_names = {"label", "class", "target", "category", "toxicity", "toxicity_label", "toxicity_level", "class_label"}
128     if not arguments.label_column and len(flags) > 1 and not any(normalized_name(column) in target_names for column in frame.columns):
129         raise ValueError("Multiple toxicity flag columns suggest multilabel data. This assignment needs one mutually exclusive class per comment. Supply an appropriate class column with --label-column; no automatic class conversion is performed.")
130     label_candidates = target_names if any(normalized_name(column) in target_names for column in frame.columns) else {"toxic"}
131     label_column = select_column(frame, arguments.label_column, label_candidates, "--label-column")
132     if text_column == label_column:
133         raise ValueError("Text and label columns must be different.")
134     print(f"\nText column: {text_column}; target column: {label_column}")
135     print(f"Duplicate comments beyond the first occurrence: {int(frame[text_column].duplicated().sum())}")
136     raw_counts = frame[label_column].value_counts(dropna=False).rename("count")
137     print("\nOriginal target distribution:\n", raw_counts.to_string())
138     raw_counts.to_csv(output / "original_class_distribution.csv")

```


Cleaning and data quality: Python code (2/2)

Exact source from the executed main.py; left-hand numbers identify original source lines.

```

139     working = frame.drop_duplicates().loc[:, [text_column, label_column]].copy()
140     working.columns = ["raw_text", "label"]
141     working.insert(0, "source_row", working.index + 2)
142     count_before = len(working)
143     working = working.dropna(subset=["raw_text", "label"]).copy()
144     print(f"Rows removed for missing text or labels: {count_before - len(working)}")
145     if pd.api.types.is_numeric_dtype(working["label"]):
146         values = working["label"].to_numpy(dtype=float)
147         if not np.all(np.isfinite(values)) or not np.all(values == np.floor(values)):
148             raise ValueError("The target appears to contain continuous scores. Supply discrete class labels for softmax classification.")
149     working["label"] = working["label"].astype(str).str.strip()
150     working["cleaned_text"] = working["raw_text"].map(clean_text)
151     empty = working["cleaned_text"].eq("") | working["label"].eq("")
152     print(f"Rows removed for empty cleaned text or blank labels: {int(empty.sum())}")
153     working = working.loc[~empty].copy()
154     conflicts = working.groupby("cleaned_text")["label"].transform("nunique").gt(1)
155     working.loc[conflicts].to_csv(output / "conflicting_labels.csv", index=False)
156     print(f"Rows removed because identical cleaned text has conflicting labels: {int(conflicts.sum())}")
157     working = working.loc[~conflicts].copy()
158     repeated = working.duplicated(subset="cleaned_text")
159     print(f"Additional repeated cleaned comments removed: {int(repeated.sum())}")
160     working = working.loc[~repeated].reset_index(drop=True)
161     counts = working["label"].value_counts().sort_index()
162     if len(counts) < 2:
163         raise ValueError("At least two distinct target classes must remain after cleaning.")
164     distribution = pd.DataFrame({"count": counts, "percentage": counts / counts.sum() * 100})
165     print("\nCleaned dataset shape:", working.shape)
166     print("\nTarget distribution:\n", distribution.to_string())
167     print("\nCleaned examples:\n", working[["cleaned_text", "label"]].head(10).to_string(index=False))
168     distribution.to_csv(output / "class_distribution.csv")
169     working.to_csv(output / "cleaned_dataset.csv", index=False)
170     return working, text_column, label_column

```

Missing values and duplicates

Check	Value
Raw rows	30000
Exact duplicate records beyond first	0
Repeated raw comments beyond first	29983
Unique raw comments	17
Cleaned unique comments	17

Missing values per column

Unnamed: 0	missing_count
Comment_ID	0
Comment_Text	0
Toxicity_Label	0
Clean	0
Toxic	0
Severe_Toxic	0
Threat	0
Identity_Attack	0
Toxicity_Score	0
Comment_Length	0
Word_Count	0
Platform	0
Topic	0
Moderation_Priority	0
Language	0

Class distribution and dataset limitation

	Raw rows	Unique cleaned comments
Clean	18579	4
Identity_Attack	1468	3
Severe_Toxic	1801	3
Threat	1459	3
Toxic	6693	4

The supplied file contains 30,000 rows but only 17 distinct raw comments. After cleaning and removal of repeated comments, 17 unique examples remain across 5 classes. Training, validation and test contain 7, 5 and 5 comments respectively. One test prediction changes accuracy by 20 percentage points. Identical cleaned text is kept in one partition only. These results demonstrate the workflow; the very small independent sample cannot establish dependable real-world accuracy.

Cleaning examples

Lowercase text retains alphabetic characters and spaces; tags, URLs, punctuation, digits, emoji and excess whitespace are removed.

raw_text	cleaned_text	label
The message targets a group unfairly and violates community rules.	the message targets a group unfairly and violates community rules	Identity_Attack
Thank you for sharing this perspective with the community.	thank you for sharing this perspective with the community	Clean
That response is aggressive and not helpful.	that response is aggressive and not helpful	Toxic
I appreciate your explanation and the discussion was helpful.	i appreciate your explanation and the discussion was helpful	Clean
I respectfully disagree but I understand your point.	i respectfully disagree but i understand your point	Clean
This article provides useful information and clear examples.	this article provides useful information and clear examples	Clean
This reply contains a threat-like statement and is unsafe.	this reply contains a threat like statement and is unsafe	Threat
This comment is rude and disrespectful.	this comment is rude and disrespectful	Toxic
This message includes threatening language and should be reviewed.	this message includes threatening language and should be reviewed	Threat
The tone of this message is toxic and inappropriate.	the tone of this message is toxic and inappropriate	Toxic

Splitting, tokenization and padding: Python code (1/2)

Exact source from the executed main.py; left-hand numbers identify original source lines.

```

173 def task_three_tokenize(frame, arguments, output):
174     print("\nTASK 3: Stratified splitting, training-only tokenization, and padding")
175     try:
176         train_validation, test = train_test_split(frame, test_size=0.2, random_state=arguments.seed, stratify=frame["label"])
177         train, validation = train_test_split(train_validation, test_size=0.2, random_state=arguments.seed, stratify=train_validation["label"])
178         if any(set(partition["label"]) != set(frame["label"]) for partition in (train, validation, test)):
179             raise ValueError("A class is absent from a partition.")
180     except ValueError as error:
181         counts = frame["label"].value_counts()
182         if counts.min() < 3:
183             raise ValueError(f"Cannot form stratified train/validation/test sets: each class needs at least three distinct comments. Counts: {counts.to_dict()}") from error
184     print("The dataset is too small for the requested proportions; allocating at least one validation and test comment per class.")
185     training_groups, validation_groups, testing_groups = [], [], []
186     for _, group in frame.groupby("label", sort=True):
187         test_count = max(1, int(np.ceil(len(group) * 0.2)))
188         remaining, test_group = train_test_split(group, test_size=test_count, random_state=arguments.seed)
189         validation_count = max(1, int(np.ceil(len(remaining) * 0.2)))
190         train_group, validation_group = train_test_split(remaining, test_size=validation_count, random_state=arguments.seed)
191         training_groups.append(train_group)
192         validation_groups.append(validation_group)
193         testing_groups.append(test_group)
194     train = pd.concat(training_groups).sample(frac=1, random_state=arguments.seed)
195     validation = pd.concat(validation_groups).sample(frac=1, random_state=arguments.seed)
196     test = pd.concat(testing_groups).sample(frac=1, random_state=arguments.seed)
197     partitions = {"train": train, "validation": validation, "test": test}
198     expected_classes = set(frame["label"])
199     if any(set(partition["label"]) != expected_classes for partition in partitions.values()):
200         raise ValueError("Every class must occur in every partition. Supply more distinct examples for the rare classes.")
201     encoder = LabelEncoder().fit(train["label"])
202     print("Class encoding:", {label: index for index, label in enumerate(encoder.classes_)})
203     tokenizer = Tokenizer(num_words=arguments.max_words, oov_token="<OOV>", filters="", lower=False)
204     tokenizer.fit_on_texts(train["cleaned_text"].tolist())
205     arrays = {}

```

Splitting, tokenization and padding: Python code (2/2)

Exact source from the executed main.py; left-hand numbers identify original source lines.

```
206     for name, partition in partitions.items():
207         sequences = tokenizer.texts_to_sequences(partition["cleaned_text"].tolist
    ())
208         inputs = pad_sequences(sequences, maxlen=arguments.max_length, padding="pos
    t", truncating="post", dtype="int32")
209         targets = encoder.transform(partition["label"]).astype("int32")
210         arrays[name] = (inputs, targets)
211         print(f"{name}: {len(partition)} comments, padded shape {inputs.shape}, cla
    ss counts {partition['label'].value_counts().to_dict()}")
212         print(f"{name} example integer sequence: {sequences[0]}")
213         print(f"{name} example padded sequence: {inputs[0].tolist()}")
214         partition[["source_row", "label"]].to_csv(output / f"{name}_rows.csv", inde
    x=False)
215         vocabulary_size = min(arguments.max_words, len(tokenizer.word_index) + 1)
216         print(f"Embedding vocabulary size: {vocabulary_size}; fixed sequence length: {a
    rguments.max_length}")
217         (output / "tokenizer.json").write_text(tokenizer.to_json(), encoding="utf-8")
218         write_json(output / "classes.json", encoder.classes_.tolist())
219         return arrays, encoder.classes_.tolist(), vocabulary_size
```

Exported execution evidence · main.py

Screenshot 14 of 27 · Generated from the executed source and saved run artifacts

Tokenizer and padded sequence output

Tokenizer vocabulary is fitted only on training comments. Unseen words use the out-of-vocabulary token; padding and truncation are applied after splitting.

```
TASK 3: Stratified splitting, training-only tokenization, and padding
The dataset is too small for the requested proportions; allocating at least one validation
and test comment per class.
Class encoding: {'Clean': 0, 'Identity_Attack': 1, 'Severe_Toxic': 2, 'Threat': 3, 'Toxic': 4}
train: 7 comments, padded shape (7, 32), class counts {'Clean': 2, 'Toxic': 2, 'Identity_Attack': 1, 'Threat': 1, 'Severe_Toxic': 1}
train example integer sequence: [12, 13, 14, 15, 3, 16, 17, 4, 6]
train example padded sequence: [12, 13, 14, 15, 3, 16, 17, 4, 6, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
validation: 5 comments, padded shape (5, 32), class counts {'Identity_Attack': 1, 'Toxic': 1, 'Severe_Toxic': 1, 'Clean': 1, 'Threat': 1}
validation example integer sequence: [3, 1, 35, 1, 37, 2, 1, 1]
validation example padded sequence: [3, 1, 35, 1, 37, 2, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
test: 5 comments, padded shape (5, 32), class counts {'Identity_Attack': 1, 'Toxic': 1, 'Severe_Toxic': 1, 'Clean': 1, 'Threat': 1}
test example integer sequence: [4, 11, 1, 1, 1, 1, 2, 41, 6, 1]
test example padded sequence: [4, 11, 1, 1, 1, 1, 2, 41, 6, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
Embedding vocabulary size: 47; fixed sequence length: 32
```


LSTM architecture and training: Python code (1/2)

Exact source from the executed main.py; left-hand numbers identify original source lines.

```

222 def make_dataset(inputs, targets, arguments, training=False):
223     dataset = tf.data.Dataset.from_tensor_slices((inputs, targets))
224     if training:
225         dataset = dataset.shuffle(len(targets), seed=arguments.seed, reshuffle_each
_iteration=True)
226     options = tf.data.Options()
227     options.threading.private_threadpool_size = 1
228     options.threading.max_intra_op_parallelism = 1
229     return dataset.batch(arguments.batch_size).with_options(options).prefetch(1)

232 def task_four_train(arrays, class_names, vocabulary_size, arguments, output):
233     print("\nTASK 4: Model architecture and training")
234     model = tf.keras.Sequential([
235         tf.keras.Input(shape=(arguments.max_length,), dtype="int32"),
236         tf.keras.layers.Embedding(input_dim=vocabulary_size, output_dim=64, mask_zer
ro=True),
237         tf.keras.layers.LSTM(64),
238         tf.keras.layers.Dropout(0.5),
239         tf.keras.layers.Dense(len(class_names), activation="softmax"),
240     ])
241     model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=0.001), loss="sp
arse_categorical_crossentropy", metrics=["accuracy"])
242     model.summary()
243     weights = compute_class_weight(class_weight="balanced", classes=np.arange(len(c
lass_names)), y=arrays["train"][1])
244     class_weights = {index: float(weight) for index, weight in enumerate(weights)}
245     print("\nTraining class weights:", class_weights)
246     checkpoint = output / "best_model.keras"
247     history = model.fit(
248         make_dataset(*arrays["train"], arguments, training=True),
249         validation_data=make_dataset(*arrays["validation"], arguments),
250         epochs=arguments.epochs,
251         class_weight=class_weights,
252         callbacks=[tf.keras.callbacks.ModelCheckpoint(str(checkpoint), monitor="val
_loss", save_best_only=True)],
253         verbose=2,
254     )
255     history_frame = pd.DataFrame(history.history)
256     history_frame.index = np.arange(1, len(history_frame) + 1)
257     history_frame.index.name = "epoch"
258     history_frame.to_csv(output / "training_history.csv")
259     figure, axes = plt.subplots(1, 2, figsize=(12, 4))
260     for axis, metric in zip(axes, ("loss", "accuracy")):
261         axis.plot(history_frame.index, history_frame[metric], label="Training")

```


LSTM architecture and training: Python code (2/2)

Exact source from the executed main.py; left-hand numbers identify original source lines.

```
262         axis.plot(history_frame.index, history_frame[f"val_{metric}"], label="Valid
    ation")
263         axis.set(xlabel="Epoch", ylabel=metric.capitalize(), title=f"Training and v
    alidation {metric}")
264         axis.legend()
265         figure.tight_layout()
266         figure.savefig(output / "training_curves.png", dpi=180)
267         plt.close(figure)
268         best_epoch = int(history_frame["val_loss"].idxmin())
269         print(f"\nUsing the checkpoint from epoch {best_epoch}, selected by validation
    loss.")
270         return tf.keras.models.load_model(checkpoint), class_weights, best_epoch
```

Exported execution evidence · main.py

Screenshot 17 of 27 · Generated from the executed source and saved run artifacts

Model architecture and training configuration

Embedding → LSTM → Dropout → Dense with softmax. Integer targets use sparse categorical cross-entropy. The checkpoint is selected using validation loss.

TASK 4: Model architecture and training

Model: "sequential"

Layer (type)	Output Shape	Param #
embedding (Embedding)	(None, 32, 64)	3,008
lstm (LSTM)	(None, 64)	33,024
dropout (Dropout)	(None, 64)	0
dense (Dense)	(None, 5)	325

Total params: 36,357 (142.02 KB)

Trainable params: 36,357 (142.02 KB)

Non-trainable params: 0 (0.00 B)

Training class weights: {0: 0.7, 1: 1.4, 2: 1.4, 3: 1.4, 4: 0.7}

Requested epochs: 15; batch size: 32; maximum sequence length: 32; vocabulary size: 47.

Command to reproduce these settings:

```
.venv/bin/python main.py --data toxic_comments_dataset.csv --epochs 15 --max-length 32 --batch-size 32 --seed 42
```

Exported execution evidence · outputs/run.log, outputs/run_config.json

Screenshot 18 of 27 · Generated from the executed source and saved run artifacts

Training and validation results for every epoch

All configured epochs were executed. Test examples were excluded from model fitting and checkpoint selection.

epoch	accuracy	loss	val_accuracy	val_loss
1	0.1429	1.6088	0.0000	1.6119
2	0.4286	1.6033	0.0000	1.6118
3	0.2857	1.5983	0.2000	1.6118
4	0.7143	1.5916	0.2000	1.6118
5	0.5714	1.5897	0.2000	1.6118
6	0.7143	1.5784	0.2000	1.6118
7	0.8571	1.5770	0.2000	1.6118
8	0.8571	1.5787	0.2000	1.6118
9	1.0000	1.5692	0.4000	1.6118
10	0.8571	1.5574	0.4000	1.6118
11	1.0000	1.5512	0.4000	1.6119
12	1.0000	1.5327	0.4000	1.6120
13	1.0000	1.5418	0.4000	1.6121
14	1.0000	1.5307	0.4000	1.6122
15	1.0000	1.5339	0.4000	1.6123

Best validation-loss checkpoint: epoch 7.

Held-out evaluation: Python code (1/2)

Exact source from the executed main.py; left-hand numbers identify original source lines.

```

273 def task_five_evaluate(model, arrays, class_names, arguments, output, best_epoch):
274     print("\nTASK 5: Held-out test evaluation and visualization")
275     test_inputs, test_targets = arrays["test"]
276     dataset = make_dataset(test_inputs, test_targets, arguments)
277     metrics = model.evaluate(dataset, return_dict=True, verbose=0)
278     probabilities = model.predict(dataset, verbose=0)
279     predictions = np.argmax(probabilities, axis=1)
280     class_ids = np.arange(len(class_names))
281     matrix = confusion_matrix(test_targets, predictions, labels=class_ids)
282     matrix_frame = pd.DataFrame(matrix, index=class_names, columns=class_names)
283     matrix_frame.index.name = "Actual"
284     matrix_frame.columns.name = "Predicted"
285     report_text = classification_report(test_targets, predictions, labels=class_id
s, target_names=class_names, digits=4, zero_division=0)
286     report = classification_report(test_targets, predictions, labels=class_ids, tar
get_names=class_names, output_dict=True, zero_division=0)
287     print(f"\nTest loss: {metrics['loss']:.4f}\nTest accuracy: {metrics['accurac
y']:.4f}")
288     print("\nConfusion matrix (rows: actual; columns: predicted):\n", matrix_frame.
to_string())
289     print("\nClassification report:\n", report_text)
290     matrix_frame.to_csv(output / "confusion_matrix.csv")
291     (output / "classification_report.txt").write_text(report_text, encoding="utf-
8")
292     pd.DataFrame({name: values for name, values in report.items() if isinstance(val
ues, dict)}).T.to_csv(output / "classification_report.csv")
293     write_json(output / "metrics.json", {
294         "test_loss": float(metrics["loss"]),
295         "test_accuracy": float(metrics["accuracy"]),
296         "macro_f1": float(report["macro avg"]["f1-score"]),
297         "weighted_f1": float(report["weighted avg"]["f1-score"]),
298         "best_epoch": best_epoch,
299         "classification_report": report,
300     })
301     prediction_frame = pd.DataFrame({"actual": np.asarray(class_names)[test_target
s], "predicted": np.asarray(class_names)[predictions]})
302     for index, name in enumerate(class_names):
303         prediction_frame[f"probability_{name}"] = probabilities[:, index]
304     prediction_frame.to_csv(output / "test_predictions.csv", index=False)
305     side = max(7, len(class_names) * 0.9)
306     figure, axis = plt.subplots(figsize=(side, side))
307     sns.heatmap(matrix_frame, annot=True, fmt="d", cmap="Blues", ax=axis, cbar=Fals
e)
308     axis.set(title="Toxic comment classification: test confusion matrix", xlabel="P
redicted class", ylabel="Actual class")
309     figure.tight_layout()
310     figure.savefig(output / "confusion_matrix.png", dpi=180)

```

Held-out evaluation: Python code (2/2)

Exact source from the executed main.py; left-hand numbers identify original source lines.

```
311     plt.close(figure)
312     print(f"\nConfusion matrix image: {output / 'confusion_matrix.png'}")
```

Exported execution evidence · main.py

Screenshot 21 of 27 · Generated from the executed source and saved run artifacts

Test metrics and classification report

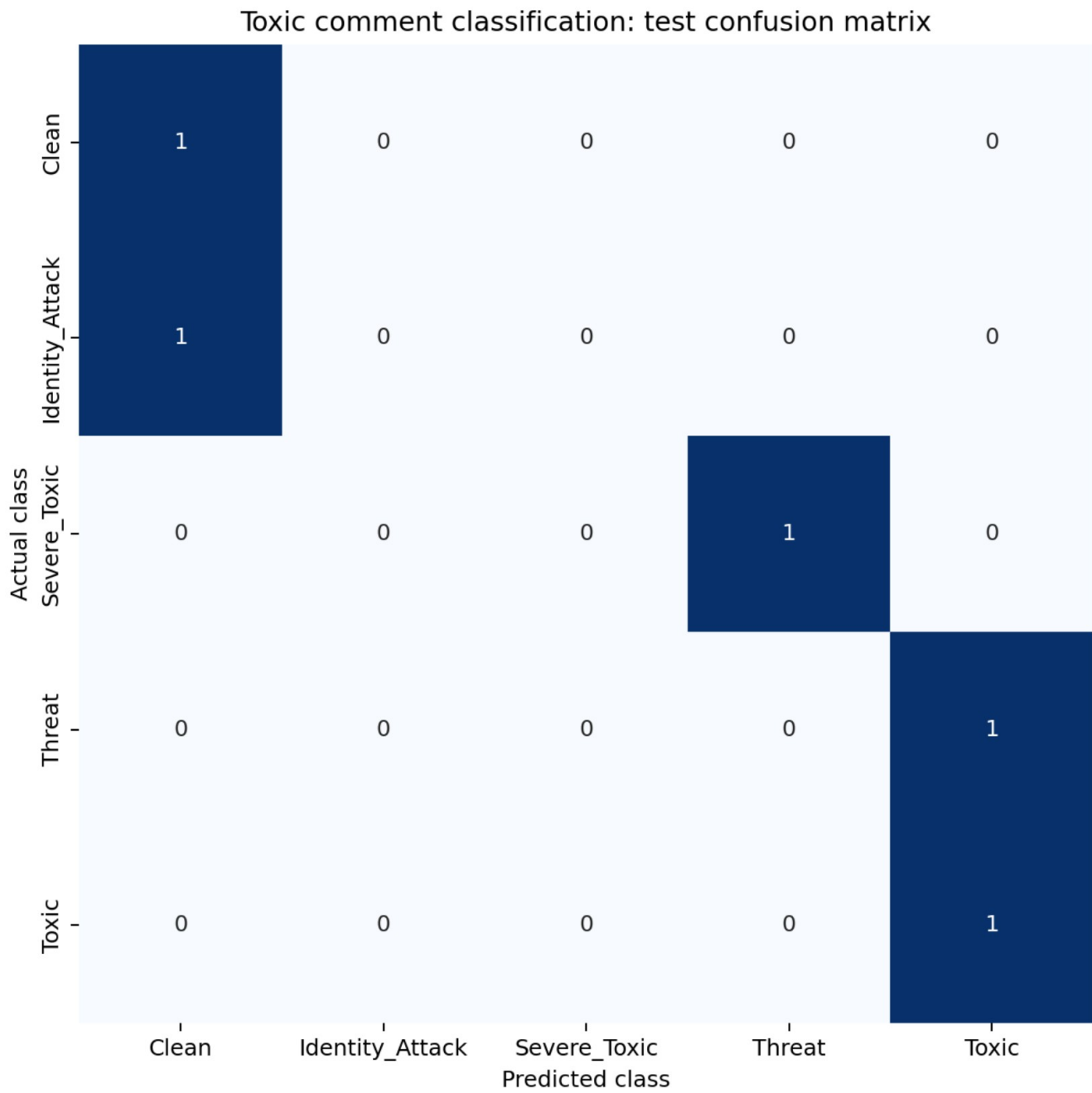
Test accuracy: 0.4000 (40.0%)
Test loss: 1.6074
Macro F1-score: 0.2667
Weighted F1-score: 0.2667
Test examples: 5

precision	recall	f1-score	support		
	Clean	0.5000	1.0000	0.6667	1
Identity_Attack		0.0000	0.0000	0.0000	1
Severe_Toxic		0.0000	0.0000	0.0000	1
Threat		0.0000	0.0000	0.0000	1
Toxic		0.5000	1.0000	0.6667	1
accuracy				0.4000	5
macro avg		0.2000	0.4000	0.2667	5
weighted avg		0.2000	0.4000	0.2667	5

The supplied file contains 30,000 rows but only 17 distinct raw comments. After cleaning and removal of repeated comments, 17 unique examples remain across 5 classes. Training, validation and test contain 7, 5 and 5 comments respectively. One test prediction changes accuracy by 20 percentage points. Identical cleaned text is kept in one partition only. These results demonstrate the workflow; the very small independent sample cannot establish dependable real-world accuracy.

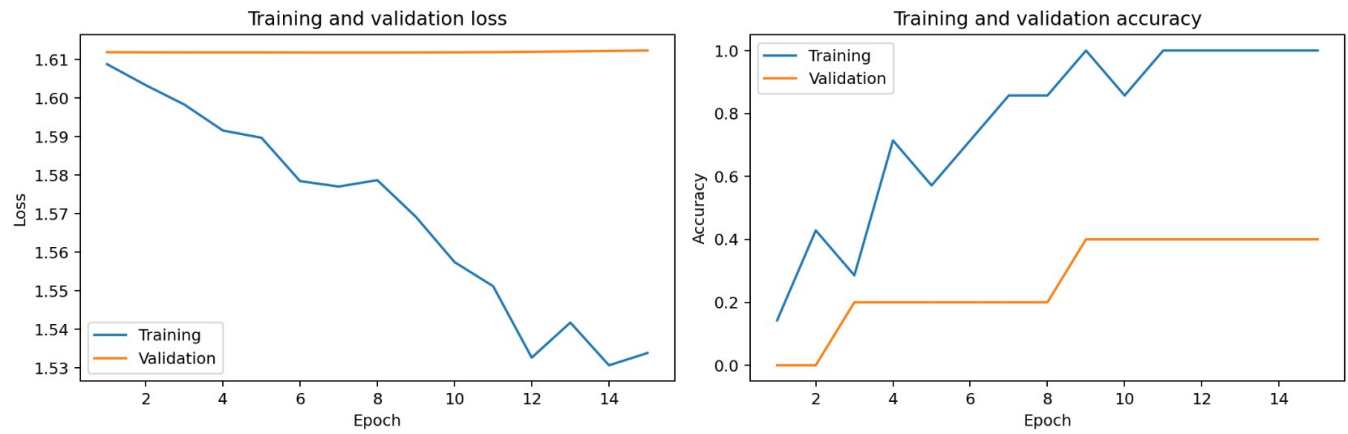
Confusion matrix on the test set

Rows show actual classes; columns show predicted classes. Counts come from the held-out test set.



Training and validation accuracy/loss curves

Curves report all training epochs. Training loss uses class weights; validation loss is unweighted. Accuracy changes in large steps because each partition contains very few distinct comments.



Individual held-out predictions

CSV row	actual	predicted
2	Identity_Attack	Clean
14	Toxic	Toxic
90	Severe_Toxic	Threat
5	Clean	Clean
12	Threat	Toxic

Each row is one independent test comment. Prediction probabilities and full metrics remain in the saved output files.

Predictions for new comments: Python code

Exact source from the executed main.py; left-hand numbers identify original source lines.

```

315 def predict_new_comments(model, class_names, arguments, output):
316     print("\nNEW COMMENTS: Predictions using the saved training vocabulary")
317     comments = arguments.predict or [
318         "Thank you for your thoughtful explanation.",
319         "Your reply is rude and insulting.",
320         "This is extremely abusive and hateful.",
321         "I will hurt you if you keep posting.",
322         "Attacking people because of their identity is unacceptable.",
323     ]
324     cleaned = [clean_text(comment) for comment in comments]
325     if any(not comment for comment in cleaned):
326         raise ValueError("A new comment is empty after cleaning; supply comments co
ntaining words.")
327     tokenizer = tokenizer_from_json((output / "tokenizer.json").read_text(encoding
="utf-8"))
328     sequences = tokenizer.texts_to_sequences(cleaned)
329     padded = pad_sequences(sequences, maxlen=arguments.max_length, padding="post",
truncating="post", dtype="int32")
330     probabilities = model(tf.convert_to_tensor(padded), training=False).numpy()
331     predictions = probabilities.argmax(axis=1)
332     oov_index = tokenizer.word_index[tokenizer.oov_token]
333     results = pd.DataFrame({
334         "comment": comments,
335         "cleaned_text": cleaned,
336         "predicted_label": [class_names[index] for index in predictions],
337         "model_probability": probabilities.max(axis=1),
338         "oov_fraction": [sequence.count(oov_index) / len(sequence) for sequence in
sequences],
339     })
340     for index, label in enumerate(class_names):
341         results[f"probability_{label}"] = probabilities[:, index]
342     results.to_csv(output / "new_comment_predictions.csv", index=False)
343     print(results[["comment", "predicted_label", "model_probability", "oov_fratio
n"]].to_string(index=False))
344     print("These comments have no supplied ground-truth labels and are not part of
test accuracy. Softmax scores are model outputs, not calibrated confidence.")

```

New-comment prediction output

comment	predicted_label	model_probability	oov_fraction
Thank you for your thoughtful explanation.	Clean	0.2050	0.3333
Your reply is rude and insulting.	Toxic	0.2030	0.5000
This is extremely abusive and hateful.	Toxic	0.2020	0.3333
I will hurt you if you keep posting.	Clean	0.2036	0.6250
Attacking people because of their identity is unacceptable.	Toxic	0.2021	0.6250

These new comments have no supplied ground-truth labels. Predictions demonstrate inference only and are excluded from the reported test accuracy.